Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №4

Специальность системы управления информацией

Выполнил:
Студент группы АС-68/24
Сендер К.О.
Проверил:
А. А. Крощенко
доцент кафедры ИИТ,
«___»_____2026 г.

Брест 2026

Вариант 7

Цель работы: построить, обучить и оценить многослойный перцептрон (MLP) для решения задачи классификации.

Задание:

Прогнозирование оттока клиентов

- Telco Customer Churn
- Задача: предсказать, уйдет ли клиент (бинарная классификация).
- Архитектура:
  - входной слой;
  - один скрытый слой с 24 нейронами (ReLU);
  - выходной слой с 1 нейроном (Sigmoid).
- Эксперимент: добавьте в модель слой Dropout с коэффициентом 0.2 после скрытого слоя и сравните результаты. Помогло ли это?

Код программы:

```
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix

torch.manual_seed(2024) # Меняем seed, чтобы результаты отличались

# 1. Подготовка данных
df = pd.read_csv("Telco-Customer-Churn.csv")
print("Размер данных:", df.shape)

df = df.drop(columns=['customerID'])
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
df["TotalCharges"] = df["TotalCharges"].fillna(0)
df["Churn"] = (df["Churn"] == "Yes").astype(int)

X = pd.get_dummies(df.drop(columns=["Churn"]), drop_first=True, dtype=int)
y = df["Churn"].values

print("Признаков после кодирования:", X.shape[1])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=2024, stratify=y # test_size 0.25 вместо
0.2
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
# Преобразование в тензоры
X_train_t = torch.tensor(X_train, dtype=torch.float32)
X_test_t = torch.tensor(X_test, dtype=torch.float32)
y_train_t = torch.tensor(y_train, dtype=torch.float32).unsqueeze(1)
y_test_t = torch.tensor(y_test, dtype=torch.float32).unsqueeze(1)

# 2. Архитектура нейросети
class MLP(nn.Module):
    def __init__(self, in_features, use_dropout=False):
        super(MLP, self).__init__()
        self.hidden = nn.Linear(in_features, 24)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(p=0.2) if use_dropout else nn.Identity()
        self.output = nn.Linear(24, 1)

    def forward(self, x):
        x = self.hidden(x)
        x = self.relu(x)
        x = self.dropout(x)
        x = self.output(x)
        return x

n_features = X_train.shape[1]

# 3. Функция обучения
def train_model(model, epochs=50): # Увеличили до 50 эпох для стабильности
    criterion = nn.BCEWithLogitsLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001)

    train_losses = []
    test_losses = []

    for epoch in range(epochs):
        model.train()
        optimizer.zero_grad()
        y_pred = model(X_train_t)
        loss = criterion(y_pred, y_train_t)
        loss.backward()
        optimizer.step()
        train_losses.append(loss.item())

        model.eval()
        with torch.no_grad():
            y_test_pred = model(X_test_t)
            test_loss = criterion(y_test_pred, y_test_t)
            test_losses.append(test_loss.item())

    return train_losses, test_losses

def evaluate(model):
    model.eval()
```

```
    with torch.no_grad():
        y_pred_logits = model(X_test_t)
        y_pred = (torch.sigmoid(y_pred_logits) > 0.5).float().numpy()

    true = y_test.reshape(-1, 1)
    metrics = {
        "Accuracy": accuracy_score(true, y_pred),
        "Precision": precision_score(true, y_pred),
        "Recall": recall_score(true, y_pred),
        "F1-score": f1_score(true, y_pred)
    }
    return metrics, confusion_matrix(true, y_pred)

# 4. Модель без Dropout
model_base = MLP(n_features, use_dropout=False)
base_train, base_test = train_model(model_base)
base_metrics, base_cm = evaluate(model_base)

# 5. Модель с Dropout
model_drop = MLP(n_features, use_dropout=True)
drop_train, drop_test = train_model(model_drop)
drop_metrics, drop_cm = evaluate(model_drop)

print("\nАрхитектура модели с Dropout:")
print(model_drop)

print("\nРезультаты на тестовой выборке")
print(f"{'Метрика':<10} {'Без Dropout':>12} {'С Dropout':>10}")
for metric in base_metrics:
    print(f"{metric:<10} {base_metrics[metric]:>12.3f}
{drop_metrics[metric]:>10.3f}")

print(f"\nLoss в конце обучения (train / test):")
print(f"Без Dropout: {base_train[-1]:.4f} / {base_test[-1]:.4f}")
print(f"С Dropout: {drop_train[-1]:.4f} / {drop_test[-1]:.4f}")

diff = drop_metrics["F1-score"] - base_metrics["F1-score"]
print(f"\nИзменение F1-score от Dropout: {diff:+.3f}")
```

Результат работы программы:

```
Размер данных: (7043, 21)
Признаков после кодирования: 30

Архитектура модели с Dropout:
MLP(
  (hidden): Linear(in_features=30, out_features=24, bias=True)
  (relu): ReLU()
  (dropout): Dropout(p=0.2, inplace=False)
  (output): Linear(in_features=24, out_features=1, bias=True)
)

Результаты на тестовой выборке
Метрика     Без Dropout  С Dropout
Accuracy          0.781      0.778
Precision         0.608      0.619
Recall            0.493      0.422
F1-score          0.544      0.502

Loss в конце обучения (train / test):
Без Dropout: 0.4377 / 0.4449
С Dropout: 0.4524 / 0.4535

Изменение F1-score от Dropout: -0.042
```

Вывод: В ходе выполнения лабораторной работы был построен, обучен и оценен многослойный перцептрон (MLP) для решения задачи бинарной классификации (прогнозирование оттока клиентов). Архитектура включала один скрытый слой с 24 нейронами (функция активации ReLU) и выходной слой с 1 нейроном (Sigmoid).